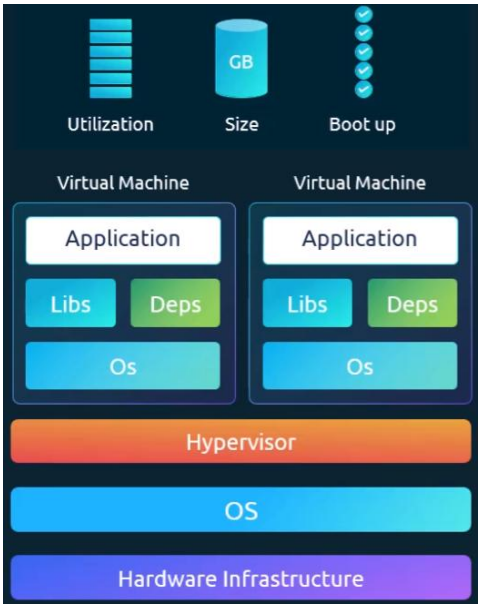
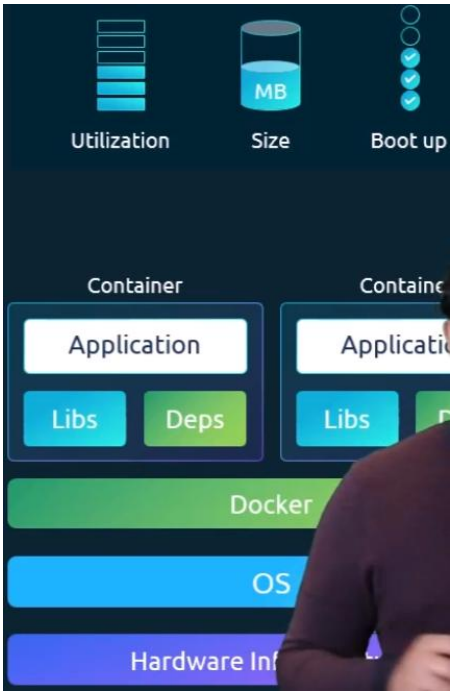
- Docker containers share the same Kernel of the host machine.
  - If the system has linux kernel, then you can't run windows container in that system.
  - The OS of the container should be compatible with the kernel of the host machine.
  - 

(virtual machine)
  - 

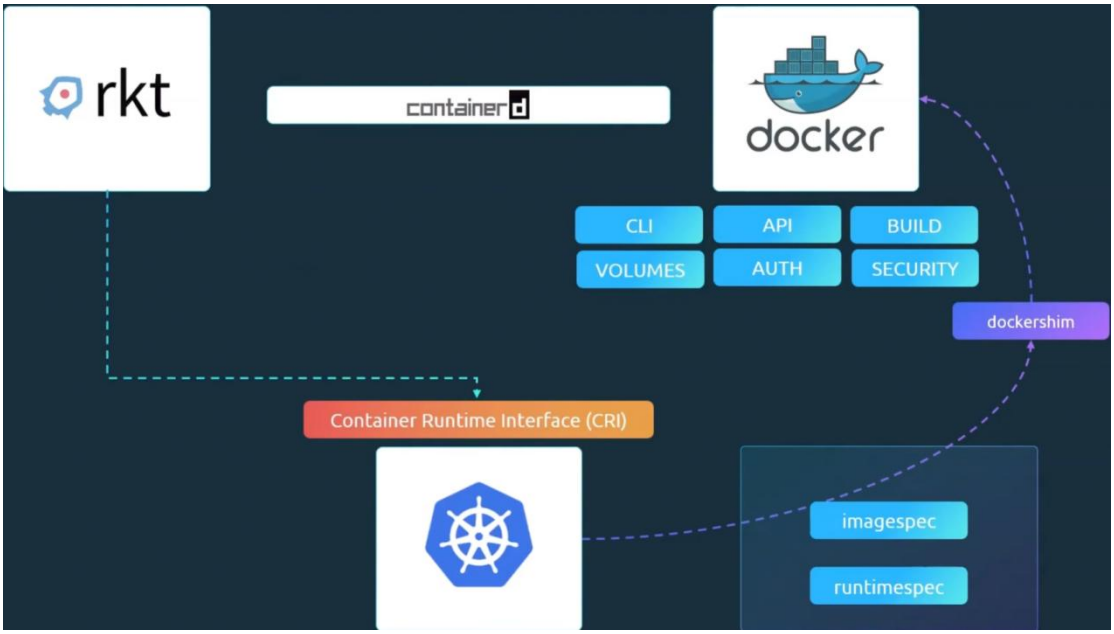
(Docker)
  -

- When you install Kubernetes in a system, then the following things are installed:
  - API Server
    - It acts as the frontend of kubernetes cluster.
  - etcd
    - Distributed key-value store that keeps the cluster information(i.e. all details about worker nodes and master nodes) of kubernetes to manage the cluster.
  - Kubelet
    - Make sure containers are running inside the nodes.
  - Container Runtime
    - Underlying software used to run software.
    - Docker has "containerd".
  - Controller
    - Brain behind the orchestration.
    - It takes decision to bring up new containers if nodes or pods goes down.
  - Scheduler
    - Implementing locks within the cluster to ensure there are no conflicts.
    - Assign the nodes to the pods.
- How does nodes are divided into master and slave?
  - Each worker node contains *Container Runtime* like containerd.
    Other container runtime are also available like *rkt, CRI-O*.
  - Master node contains ***kube-apiserver*** and Worker node contains ***kubelet*** , this kube-apiserver makes the node master.
  - ***Kubelet*** agent of the worker node connects with the master node to do the things.
- Basic commands:
  - **kubectl run hello-minikube**
    kubectl run command is used to deploy an application to the cluster.
  - **kubectl cluster-info**
    it gives the information about the **cluster**.
  - **kubectl get nodes**
    it list all the nodes of the cluster.
- Docker vs containerd
  - Kubernetes introduced an interface called CRI (container runtime interface)
  - CRI allowed every vendor to work as a container runtime for kubernetes as long as they follow the OCI (open container initiative) standard.

- This CRI has 2 specs:
  - imagespec

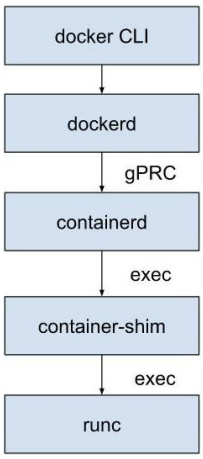
    it is the specification about how an image should be built.
  - runtimescape

    it is the specification about how any container runtime should be developed.
- Docker was there way before CRI was introduced. So, Kubernetes introduced ***dockershim*** to use Docker which was a temporary solution.
- Now, *Docker Runtime* (containerd) contains inbuilt CRI.



- Docker has multiple things that work like docker cli, api, volumes etc etc.
- **containerd** is CRI compatible, can be used as a container runtime separately with Kubernetes.
-

NOTE about Docker architecture layers ----------



- When you run **docker run nginx**, internally the below happens
  - **Docker CLI** sends request to **dockerd**.
  - dockerd *(docker daemon)* does image pull, container setup, volume setup, container config.

    dockerd is the brain of docker.

    Then, it calls **containerd**.
  - containerd calls **runc**.

    It actually creates a contaienr (namespace, cgroups)
- docker = dockerd (manager) + containerd (executor) + runc (low-level runner)
- The parts of docker:
  - Docker CLI
    - the **docker** command you type.
    - It talks to the docker daemon (**dockerd**) via REST API.
  - Docker Daemon (**dockerd**)
    - Its brain of Docker.
    - It handles containers, images, networks, volumes.
    - Exposes API for CLI/Tools.
  - Docker API
    - REST API used by CLI, or external tools.
    - Ex: Docker Desktop, CI tools
  - Docker Build System
    - Builds images from *Dockerfile*.
    - Internally uses ***Buildkit*** (modern Docker).
  - Docker objects (managed by Docker)
    - Containers, volumes, networks, images.

- The parts outside of Docker.
  - **Container Runtime** (high level)
    - containerd, it manages pulling images, container lifecycle.
    - Originally part of docker, now a separate CNCF project.
  - **Low Level Runtime**
    - **runc**
    - Actually creates containers using namespace, cgroups.
  - ---

- How images are built?
  - Docker is something that calls ***containerd*** which further calls ***runc*** to create an image.
  - Any vendor can use *containerd* and create image.
  - It doesn't mean that docker is just a tool like others. It's a complete platform which contains
    - CLI (docker)
    - Daemon (dockerd)
    - Image build system (buildkit)
    - Networking, Volumes, Image management
  - It internally uses **containerd** and **runc**.
  - **containerd**: pull, start, stop the container

    runc: actually runs the container (namespaces, cgroups)
  - Before

    kubernetes -- dockershim -- Docker -- containerd -- runc
  - Now

    kubernetes -- containerd -- runc
  - Kubernetes removed docker completely from its pipeline.
  - You just build the image and push in the registry, rest of the things will be done by Kubernetes.

    Because anyways it uses ***containerd*** which can pull and delegate the call to ***runc*** to run the container.

    Docker now adapted to CRI standard, so it builds CRI standard images that can be directly used by Kubernetes.
  - Docker is having Docker Networks, Volumes, Lifecycle, How will they all be managed without Docker in Kubernetes?
    - For networking, Kubernetes has **CNI** which is very powerful.

- For Storage, PersistentVolumes, PersistentVolumeClaims, StorageClasses are there.
- For lifecycle, pods and replica management is there.

- Kubernetes has **crictl** to talk to containerd in runtime for debugging purpose.
  - Docker doesn't use *crictl* to talk to containerd (don't get confused on this)
  - Both **dockerd** and **crictl** talks to *containerd* but in different ways.
    - *dockerd* talks to *containerd* via its own API (gRPC)
    - *crictl* talks to *containerd* via CRI plugin (*CRI API*)
  -
```
docker CLI = developer tool (build + run)
crictl     = debugging tool (inspect runtime)
```
  -

docker cli	crictl	Description	Unsupported Features
attach	attach	Attach to a running container	--detach-keys , --sig-proxy
exec	exec	Run a command in a running container	--privileged , --user , --detach-keys
images	images	List images	
info	info	Display system-wide information	
inspect	inspect , inspecti	Return low-level information on a container, image or task	
logs	logs	Fetch the logs of a container	--details
ps	ps	List containers	
stats	stats	Display a live stream of container(s) resource usage statistics	Column: NET/BLOCK I/O, PIDs
version	version	Show the runtime (Docker, ContainerD, or others) version information	

  - **crictl** can connect to any *CRI* configured runtime, not only *containerd*.

- **Minikube**
  - It provides a single node environment where all the processes of both control plane and data plane are present in a single node.
  - 

  - it provides an iso image directly that can be run using any virtual box like VM ware, virtualbox ..etc.
  - **minikube start --driver=virtualbox** (use --driver to run minikube inside virtual machine)

    if you are using WSL then virtualbox vms will not work because it cannot directly contact the hyperv, so, instead give the --driver=docker.

    If you remove WSL then Docker will not work.
- **Pods**
  - The Nodes doesn't run the application directly. Rather the container is wrapped with an object called as **pod**.
  - It is the smallest object that can be created in Kubernetes.
  - Do not add multiple containers inside a single pod if you want to scale, rather create different pods for that.

    And for CPU or other usage, then run the pods in multiple nodes.
  - Its not a restriction that we can create only one container inside a pod, we can create many.

    But, those containers shouldn't be of same kind.

    There are use-cases that create helper containers along with the application container inside the same pods which lives and dies along with the main application container.
  - The application and helper containers can communicate with eachother.
- Why Helper containers?
  - Lets see only docker containers for now.

- You have some application containers. But as your app grows, you need to process some data.

  So, you created some other containers to process the respective containers.
- Now, you need to keep shared volumes for each application and helper container pairs.

```
docker run python-app
docker run python-app
docker run python-app
docker run python-app
docker run helper --link app1
docker run helper --link app2
docker run helper --link app3
docker run helper --link app4
```

  (--link is the legacy method to link one container to another in docker)

  Now, you need to keep track of the containers like

App	Helper	Volume
Python1	App1	Vol1
Python2	App2	Vol2

- With ***Pods***, Kubernetes does all these by default.

  All the containers inside a Pod can access each other, can access the same volume, and created and destroyed together.
- Multiple containers are very rare use-case.

➢ To create a **Pod**

```
C:\Users\alokr>kubectl run nginx --image=nginx
pod/nginx created

C:\Users\alokr>kubectl get pods
NAME    READY   STATUS              RESTARTS   AGE
nginx   0/1     ContainerCreating   0          6s

C:\Users\alokr>kubectl get pods
NAME    READY   STATUS              RESTARTS   AGE
nginx   0/1     ContainerCreating   0          13s

C:\Users\alokr>kubectl get pods
NAME    READY   STATUS    RESTARTS   AGE
nginx   1/1     Running   0          27s
```

- **kubectl run <name> --image=<image name>**

  it'll pull the image from docker registry and create a pod and run the container inside that.

```
C:\Users\alokr>kubectl describe pod nginx
Name:             nginx
Namespace:        default
Priority:         0
Service Account:  default
Node:             minikube/192.168.49.2
Start Time:       Thu, 23 Apr 2026 03:12:38 +0530
Labels:           run=nginx
Annotations:      <none>
```

- **kubectl describe pod <name>**

  it displays a lot of details about the pod.

  Also, one more command is there **kubectl get pods -o wide**

```
C:\Users\alokr>kubectl get pods -o wide
NAME    READY   STATUS    RESTARTS   AGE    IP           NODE       NOMINATED NODE   READINESS GATES
nginx   1/1     Running   0          7m6s   10.244.0.3   minikube   <none>           <none>
```

Pods with YAML

```yaml
apiVersion: v1
kind: Pod
metadata:
  name: myapp-pod
  labels:
    app: myapp
    type: front-end
spec:
  containers:
  - name: nginx-container
    image: nginx
```

- **apiVersion**
  - It's the Kubernetes API version, not your app's version.

    Because the app version will be there in the docker image like *my-app:v1* .
  - **Kubernetes API** and **kube-apiserver** both are different.

    kube-apiserver exposes the *Kubernetes API* .

    analogy: kube-apiserver: nodejs application, kubernetes API: the exposed REST end-points.
  -
    ```yaml
    apiVersion: v1
    kind: Pod
    ```

    internally it becomes: **/api/v1/pods**
  -

API Version	Used For	Resources
`v1`	Core/basic resources	Pod, Service, ConfigMap, Secret
`apps/v1`	Workload controllers	Deployment, ReplicaSet, StatefulSet, DaemonSet
`batch/v1`	One-time / scheduled tasks	Job, CronJob
`networking.k8s.io/v1`	Networking	Ingress, NetworkPolicy
`rbac.authorization.k8s.io/v1`	Permissions & access control	Role, RoleBinding, ClusterRole
`autoscaling/v2`	Auto scaling	HorizontalPodAutoscaler
`storage.k8s.io/v1`	Storage management	StorageClass, VolumeAttachment

- **kind**
  - Its like what kind of object do you want to create.

- Simple analogy:

  apiVersion: which rule you want to apply?

  kind: applying that rule, what do you want to do?

kind	Meaning
`Pod`	Run a container
`Deployment`	Manage multiple Pods
`Service`	Expose Pods
`ConfigMap`	Store config
`Secret`	Store sensitive data
`Job`	Run task once

- **metadata**
  - There are some specific keys like *name, labels, namespace, annotations ..etc* which kubernetes expects to be inside metadata, other than those accepted keys, you shouldn't give anything.
  - Under ***labels***, you can give your own key-value pairs you want.
- **specs**
  - spec = what you want Kubernetes to actually do (the desired state)
  - Specs defines: which container? which image? how it should run?
  - In simple terms: ***specs** is the desired state. **Controllers** (inside controller-manager) work to make actual state match it*.
  - You can see, ***containers*** is a list.

    If you give multiple container name and image, then it'll run that many containers inside a single pod.

    ```
    containers:
      - name: app
        image: nginx
      - name: sidecar
        image: busybox
    ```

  - Deployment controller → ensures correct number of Pods

    Node controller → handles node health

    ReplicaSet controller → maintains replicas

- Inside *containers*, you'll give list of dictionaries each containing *name* and *image*.

  Image: the actual container image that is pulled from registry.

  Name: just a name of that container inside Kubernetes.

- At the end, run the command

  kubectl create -f <yml file name>

  kubectl apply -f <yaml file name>

  create doesn't update, just create the pods if doesn't exist. If exist then error.

  apply update the existing config (if present) or create the new configs (if not present) according to yaml. If you change yaml further, then you can again call *apply* but not *create*.

  Always use **apply** instead of *create*.

- **kubectl describe pod <pod name>**

  it displays all the details about the pod.

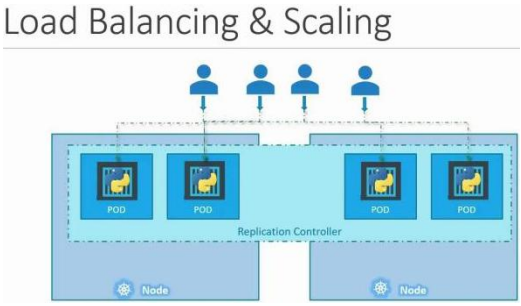
- **kubectl run nginx --image nginx**

  you can create pods using **run** command as well, if you want to debug something or create a pod quickly then it can be used.

  apply or **create** gives control because you can configure the things in yaml or any other file.

➢ F

Replication Controllers and ReplicaSets

- Controller Manager (CM) consists of different types of controllers, one of them is ***Replication Controller***.
- **Replication Controller**
  - It keeps the desired number of pods running.
  - If one pod goes down, it build another to keep the desired state.
  - Does that mean, if you want only one then you can't use Replication Controller? NO, if that single pod goes down, then Replication Controller build another pod in same or different node to keep the application running.
  - It also helps in scaling by building pods through out the cluster.
  - 

  - ```yaml
rc-definition.yml
apiVersion: v1
kind: ReplicationController
metadata:
  name: myapp-rc
  labels:
    app: myapp
    type: front-end
spec:
  replicas: 3
  template:
    metadata:
      name: myapp-pod
      labels:
        app: myapp
        type: front-end
    spec:
      containers:
      - name: nginx-container
        image: nginx
```
    - It is confusing that, when the kind was '*Pod*', we were writing *containers* inside the *spec* and that's it.
```

- But here, there is a *spec* inside *spec*.

- Here, we are not using ***Pods*** directly, instead we are telling Kubernetes to create the pods.
- First level is for *Replication Controller*, which will manage the pods.

  So, inside that, we are giving the template of the *Pod*.
- That ***template*** key inside the Spec will be exact same as the yaml configuration of kind *Pod*.

  The difference is, inside that template you don't write *apiVersion*, *kind*.

```
ReplicationController
└── spec (RC config)
    ├── replicas: 3
    └── template
        └── spec (Pod config)
            └── containers
```

- Run the same command i.e.

  kubectl apply/create -f <yaml file name>

  to run the *Replication Controller*.
- You can see in the yaml, the replication controller's name is *myapp-rc* (against metadata.name), so all the pods will have name with prefix *myapp-rc*.

➢ **ReplicaSets**

- Replication Controller and ReplicaSets have same purpose, but Replication Controller is older way to handle this. Nowadays, ReplicaSets is widely used.
- ReplicaSets is there in ***apps/v1*** (previously we were using *v1*)
- All the things are same in both *ReplicaSets* and *Replication Controller* yaml configs.

  Just, one extra key ***selector*** (under spec i.e. parent spec, not inside template's spec) will be there.

  [it is sibling of template, replicas]
  - This is mentioned to tell the ReplicaSet that which Pods come under this.
  - But, why is this required where we are already giving the Pod's template inside it?

    It is because if the Pods are already created before and you want them to come under this.
- **selector** works mainly on labels, means it'll select the pods according to the key and values inside respective *labels* of the pods.

- ***matchLabels*** and ***matchExpressions*** are there.
  -
```
matchLabels:
  app: myapp
```
    it'll check for the exact match, i.e. Pods having value 'myapp' against the key 'app' inside their *labels* will be selected.
  -
```
matchExpressions:
  - key: env
    operator: In
    values:
      - prod
      - staging
```
    it is advanced filtering. Pods having value 'prod' or 'staging' against the key 'env' inside their *labels* will be selected.
- **NOTE:**

  Selectors don't work only for Pods — they work for different resources, but they always select based on labels.
- Replication Controller also has this *selectors* options but user input is not mandatory there, if its not mentioned then it'll take all the pods whose labels are matching with the one written in the yaml config of this replication controller's template.
- Lets say you gave ***replicas*** as 4, and there already 4 pods of same type running. then when you give the *selector*, it'll not create any other pod because already 4 are there. So instead of creating new pods, it'll manage those.

  And if any pod among them goes down, it'll create one more.
- If you have configured some replicas and its running, and want to scale now. Then you'll have the below options
  - Update the yaml and run *kubectl apply* command.

    kubectl apply -f <yaml file name>
  - Run the below command without updating the yaml file.

    kubectl scale --replicas=6 -f <yaml file name>

    [NOTE: It'll not update the yaml file to replica = 6]
  - Or the below command without mentioning the yaml file.

    kubectl scale --replicas=6 replicaset <existing replicaset name> (in our case, replicaset name will be *myapp-replicaset*)

(that, replicaset in between is the type, what do you want to scale)

```
kubectl create -f replicaset-definition.yml
```

```
kubectl create -f replicaset-definition.yml
```

terminal

```
$ kubectl delete replicaset myapp-replicaset
```

*Also deletes all underlying PODs

```
$ kubectl replace -f replicaset-definition.yml
```

```
kubectl scale -replicas=6 -f replicaset-definition.yml
```

NOTE ----------

- The ***labels*** inside the ***selector*** and ***labels*** inside the ***template*** (i.e. Pod) must be same.
- Otherwise, when you run, even if it creates the pod with the labels defined inside template, it'll not match with the *selector*'s *labels*.
  So, it'll think that those pods are not its own, and it'll keep creating the pods.
- However, currently it'll give error saying *selector's* labels and *template's* labels are not matching.
- Template: creation of pods
  Selector: owning those pods
- Labels of the replicaset can be anything, that doesn't need to be matched.

```yaml
kind: ReplicaSet
metadata:
  name: myapp-replicaset
  labels:
    app: myapp-rs
spec:
  replicas: 3
  selector:
    matchLabels:
      env: production  # 1
  template:
    metadata:
      name: nginx-2
      labels:
        env: production # 2
    spec:
      containers:
        - name: nginx
          image: nginx:alpine
```

- See the below

```yaml
apiVersion: v1
kind: Pod
metadata:
  name: nginx-2
  labels:
    env: production
spec:
  containers:
    - name: nginx
      image: nginx:alpine
```

```yaml
apiVersion: apps/v1
kind: ReplicaSet
metadata:
  name: myapp-replicaset
  labels:
    app: myapp-rs
spec:
  replicas: 3
  selector:
    matchLabels:
      env: production  # 1
  template:
    metadata:
      name: nginx-2
      labels:
        env: production # 2
    spec:
      containers:
        - name: nginx
          image: nginx:alpine
```

- I had 2 yamls, one for **Pod** and the other for **ReplicaSet**
- I ran one pod using that Pod yaml.

```
$ kubectl get pods
NAME      READY   STATUS    RESTARTS   AGE
nginx-2   1/1     Running   0          3s
```

- You can see, Pod's labels and ReplicaSet's selector's labels are matching. Also, ReplicaSet's selector's labels and template's labels are matching.
- Now, when I run the replicaset using this yaml, it'll create 2 more pods. Because required is 3, and 1 is already running (nginx Pod script).

```
$ kubectl get pods
NAME                      READY   STATUS    RESTARTS   AGE
myapp-replicaset-c2hx7    1/1     Running   0          4s
myapp-replicaset-hl76d    1/1     Running   0          4s
nginx-2                   1/1     Running   0          26s
```

- The pods started from replicaset are having *myapp-replicaset* as the prefix, because the replicaset's name is *myapp-replicaset*.

- Now, if I delete that nginx-2 pod, then it should create one more.

```
$ kubectl delete pod nginx-2
pod "nginx-2" deleted from default namespace
```

```
NAME                      READY   STATUS    RESTARTS   AGE
myapp-replicaset-c2hx7    1/1     Running   0          6m11s
myapp-replicaset-hl76d    1/1     Running   0          6m11s
myapp-replicaset-sh8cm    1/1     Running   0          5s
```

- Now, if you create one pod using that Pod yaml file, it'll be created. But it'll be deleted by our replicaset immediately because the required number of pods is 3, not 4.

- NOTE
  - Lets say one pod is having n number of key value pairs in its labels like

```yaml
apiVersion: v1
kind: Pod
metadata:
  name: nginx-2
  labels:
    env: production
    key1: value1
    key2: value2
spec:
```

  - And, in the replicaset we are writing 2 key value pairs like

```yaml
apiVersion: apps/v1
kind: ReplicaSet
metadata:
  name: myapp-replicaset
  labels:
    app: myapp-rs
spec:
  replicas: 3
  selector:
    matchLabels:
      env: production
      key1: value1
  template:
```

  - Now, all the Pods having labels both **env: production** and **key1: value1** will be selected.

    If any Pod contains only 1 of these or none of these then it'll not be selected.

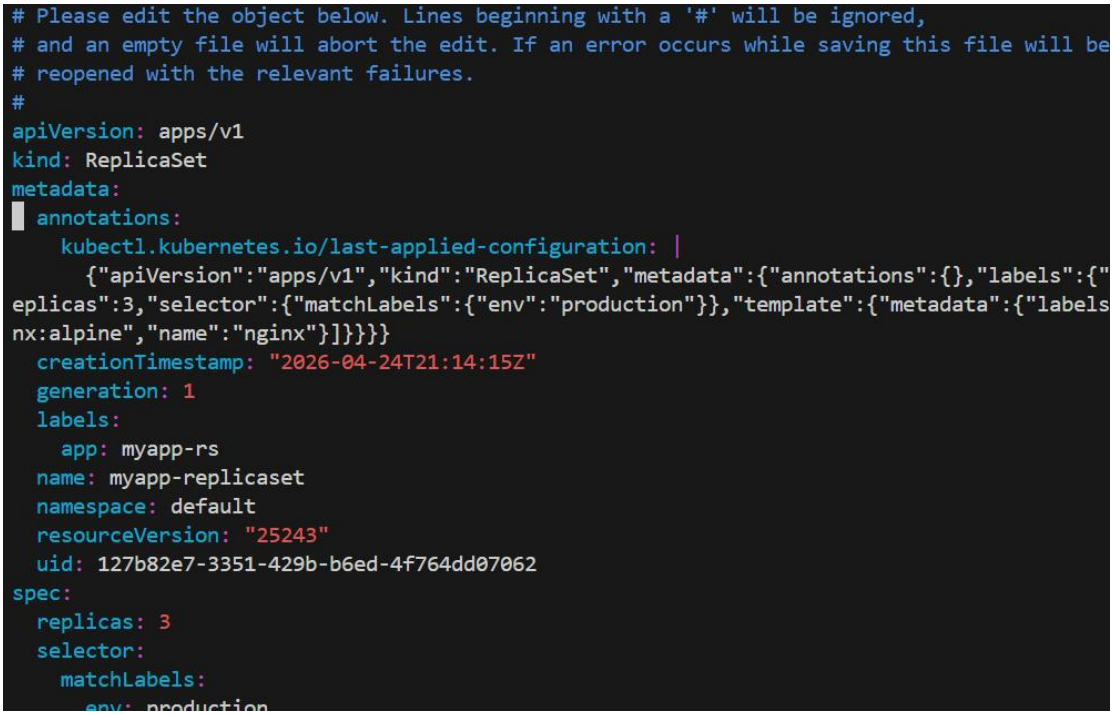
  - Simply, the selector works on **'AND'** concept, not 'OR'.

- If you execute the command

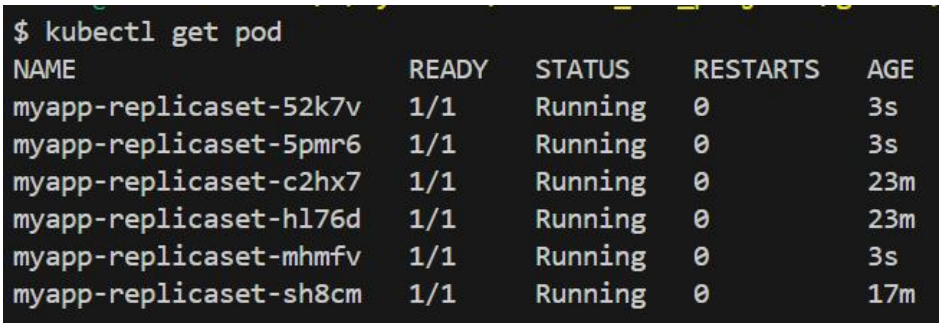
  kubectl edit replicaset <replicaset name**>**

  it'll open a yaml file (that is being created in memory for the currently running replicaset) and you can edit there. It'll directly reflect on the running services.

  It'll look something like this



- I updated the ***replicas*** to 6, and now 6 pods are running



- NOTE: it'll not change our yaml file. This yaml file is a temporary version that is being there in memory still the replicaset is running (or not deleted).
  once the replicaset is gone, this temporary yaml will also go.

Deployments

- Consider the below scenario
  - You have a *web server* to deploy.
  - You want it to have multiple instances to distribute the load properly and to have high availability.
  - When a new version of the application present in the Docker registry, you want to upgrade the instances.

    But not at once, it should be upgraded ONE-BY-ONE. This process is called ***Rolling Update***, to minimize user disruption.

    Without *Rolling Update* (i.e. upgrade one-by-one) user will face downtime.
  - In case an update introduces an Error, a quick **rollback** mechanism is essential.
  - If you want to make multiple changes
  - There is something called ***rollout***, when you change anything inside pod template i.e.

    Docker image, Environment variables, CPU/Memory limits, Label inside template .. then Kubernetes creates a new ReplicaSet and gradually shifts the pods from older ReplicaSet to newer one.
  - Lets say you changes something (which may trigger rollout), but you don't want to rollout on every changes. You should be able to *pause* and *resume* this rollout.

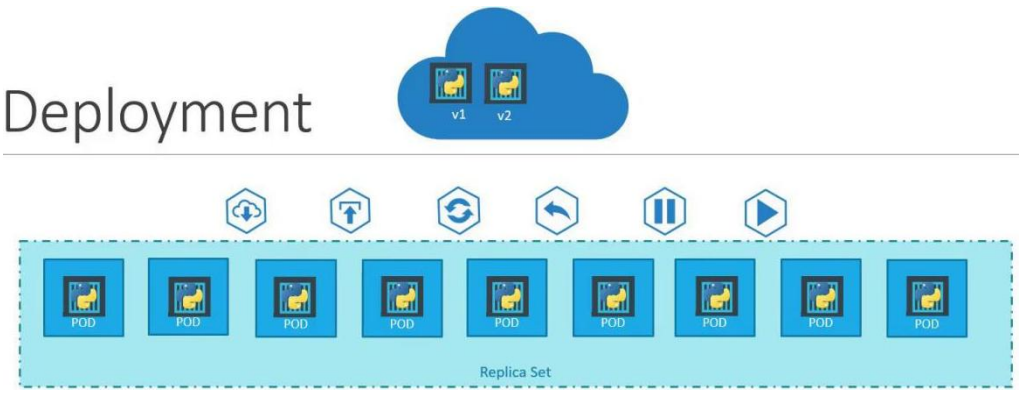
    You must have control on this.

    You can do this in Kubernetes.

    For example:

    change image: rollout starts

    change env variables: rollout starts

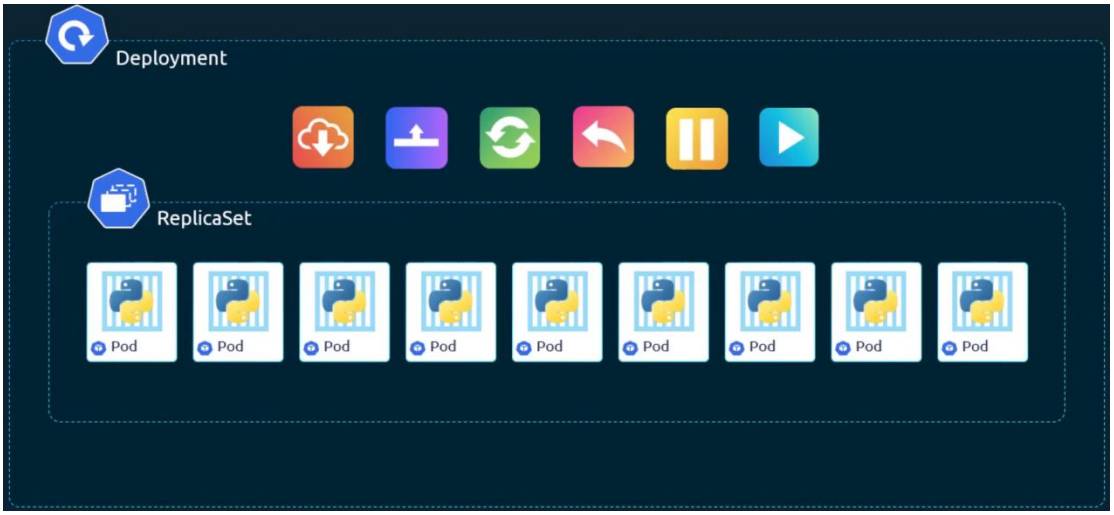
    this causes multiple unnecessary rollouts, what you want is to pause this, then you change all the things, then the rollout should happen.
  - 

- What is Deployment?
  - As we saw earlier, first we were creating Pod by ourselves which is not proper.
    We want someone to manage the pods and create if any pod goes down.
    So, we used ReplicaSet.
  - Now, for *rollback* and *rolling update* also we want someone to handle the *ReplicaSet*.
    Means, when anything needs to be changed, then someone should create a new ReplicaSet with the updated template.
    And it should gradually scale down the pods of older ReplicaSet and scale up the pods of new ReplicaSet to ensure ZERO Downtime.
  - So, here, **Deployment** is used.
    When anything changed [i.e. Docker image, Environment variables, CPU/Memory limits, Label inside template], it creates a new ReplicaSet.
  - The balances between Old ReplicaSet and New ReplicaSet happens like below lets say you are having 10 replicas.

```
Step 1:
Old RS = 9
New RS = 1

Step 2:
Old RS = 8
New RS = 2

Step 3:
Old RS = 7
New RS = 3

...

Final:
Old RS = 0
New RS = 10
```

    - Actually it doesn't increase and decrease the pods like this I mean decreasing or increasing by 1, sometimes it does by 2 or 3 as well depending upon **maxUnavailable** (max number of pods which are up) and **maxSurge** (max number of pods that can be down)
    - maxUnavailable = 25%, it means at an point the total number of pods (old replicaset's pods count + new replicaset's pod count) can be at least 7.25==8 (if total replicas = 10)

- maxSurge = 25%, it means at an point the total number of pods can be maximum 12.5==13 (if total replicas = 10)



- Pull, upgrade, rolling update, rollback (undo), pause (rollout pause), resume (rollout resume

- YAML of Deployment
  - Whatever thing was there in ReplicaSet yaml, all will be same, but the ***kind*** will be now Deployment.
  - Inside ***spec***, along with template, replicas, selector ..etc, a few more fields will be there i.e.

    strategy, ***rollingUpdate***,

```
spec:
  replicas: 3

  selector:
    matchLabels:
      app: myapp

  strategy:
    type: RollingUpdate   # or Recreate
    rollingUpdate:
      maxUnavailable: 1
      maxSurge: 1

  minReadySeconds: 5
  revisionHistoryLimit: 10
  progressDeadlineSeconds: 600
  paused: false
```

  (outer ***specs***)

- How the naming of the deployments, replicasets and pods are given?
  - metadata.name (deployment's name, let say *my-dep*)

- Name of replicaset created from this deployment will have name in the format *my-dep-58c87667cd* (some random suffix)
- Name of the pods created from that replicaset *my-dep-58c87667cd-n2694* (again some random suffix, that's why name of the template is not required, its ignored anyways)

```
$ kubectl get all
NAME                                    READY   STATUS    RESTARTS   AGE
pod/myapp-deployment-58c87667cd-n2694   1/1     Running   0          11m
pod/myapp-deployment-58c87667cd-rm4bg   1/1     Running   0          11m
pod/myapp-deployment-58c87667cd-smkkv   1/1     Running   0          11m

NAME                 TYPE        CLUSTER-IP   EXTERNAL-IP   PORT(S)   AGE
service/kubernetes   ClusterIP   10.96.0.1    <none>        443/TCP   2d22h

NAME                                READY   UP-TO-DATE   AVAILABLE   AGE
deployment.apps/myapp-deployment    3/3     3            3           11m

NAME                                           DESIRED   CURRENT   READY   AGE
replicaset.apps/myapp-deployment-58c87667cd    3         3         3       11m
```

-

```yaml
apiVersion: apps/v1
kind: Deployment
metadata:
  name: myapp-deployment
  labels:
    tier: frontend
    app: dep-nginx
spec:
  selector:
    matchLabels:
      app: myapp
  replicas: 3
  template:
    metadata:
      # name is ignored when created by
      #       deployment or replicaset
      name: myapp-pod
      labels:
        app: myapp
    spec:
      containers:
        - name: nginx
          image: nginx:alpine
```

- **kubectl get all**

  it'll display all the objects created.

➢ **Rollout and Versioning**

- When you first create a Deployment, it triggers a ***rollout***.
- A new rollout creates a new ReplicaSet which is recorded as a new deployment version.
- Each rollout has a version which is helpful to ***rollback*** to a specific version if anything fails during the update.

- The below command displays the current ***rollout*** progress (live status)

  kubectl rollout status deployment.apps/<deployment name**>**

  (or) **kubectl rollout status deployment <**deployment name**>**

```
$ kubectl rollout status deployment.apps/myapp-deployment
deployment "myapp-deployment" successfully rolled out
```

  (or)

```
$ kubectl rollout status deployment myapp-deployment
deployment "myapp-deployment" successfully rolled out
```

  [deployment myapp-deployment is same as **deployment.apps/myapp-deployment]**

- The below command displays the past versions, i.e. previous deployments.

  Each change = new revision

  kubectl rollout status deployment.apps/<deployment name**>**

  (or) **kubectl rollout status deployment <**deployment name**>**

```
$ kubectl rollout history deployment.apps/myapp-deployment
deployment.apps/myapp-deployment
REVISION  CHANGE-CAUSE
1         <none>
```

  (or)

```
$ kubectl rollout history deployment myapp-deployment
deployment.apps/myapp-deployment
REVISION  CHANGE-CAUSE
1         <none>
```

  you can see here CHANGE-CAUSE is <none> because you need to tell Kubernetes explicitly to record the cause of changes.

  So, you need to write **--record** flat after the *kubectl apply* or *kubectl create* or *any command that will lead to create a revision …. --record* .

➢ **Deployment Strategy**

- If you want to update all the pods to latest then

  one way is, delete all the current running pods and create new pods having the updated version of your application.

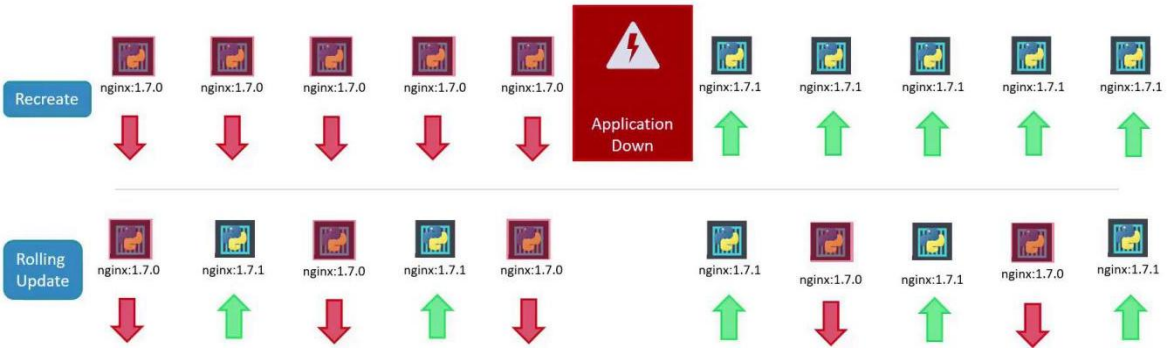
  This strategy is ***Recreate***.

  - The problem is: it'll make the application down.

- So, ***RollingUpdate*** is the default deployment strategy.

  Here, one older pod will be deleted and one new pod will be created.

  So that, the application doesn't go down.



- If you want to *rollback* to the previous revision

  kubectl rollout undo deployment/<deployment name>

- Demo of Rollout
  - I created a *Deployment* using a yaml file.

```
$ kubectl rollout status deployment myapp-deployment
deployment "myapp-deployment" successfully rolled out
```

```
$ kubectl rollout history deployment myapp-deployment
deployment.apps/myapp-deployment
REVISION  CHANGE-CAUSE
1         kubectl.exe create --filename=kube_deployment.yaml --record=true
```

you can see the CHANGE-CAUSE because of the flag ***--record=true***

  - Now, I changed the container's image and executed *kubectl apply* .. command.

```
$ kubectl rollout status deployment myapp-deployment
Waiting for deployment "myapp-deployment" rollout to finish: 3 out of 6 new replicas have been update
Waiting for deployment "myapp-deployment" rollout to finish: 3 out of 6 new replicas have been update
Waiting for deployment "myapp-deployment" rollout to finish: 3 out of 6 new replicas have been update
Waiting for deployment "myapp-deployment" rollout to finish: 3 out of 6 new replicas have been update
Waiting for deployment "myapp-deployment" rollout to finish: 3 out of 6 new replicas have been update
Waiting for deployment "myapp-deployment" rollout to finish: 4 out of 6 new replicas have been update
Waiting for deployment "myapp-deployment" rollout to finish: 4 out of 6 new replicas have been update
Waiting for deployment "myapp-deployment" rollout to finish: 4 out of 6 new replicas have been update
Waiting for deployment "myapp-deployment" rollout to finish: 4 out of 6 new replicas have been update
Waiting for deployment "myapp-deployment" rollout to finish: 5 out of 6 new replicas have been update
Waiting for deployment "myapp-deployment" rollout to finish: 5 out of 6 new replicas have been update
Waiting for deployment "myapp-deployment" rollout to finish: 5 out of 6 new replicas have been update
Waiting for deployment "myapp-deployment" rollout to finish: 5 out of 6 new replicas have been update
Waiting for deployment "myapp-deployment" rollout to finish: 5 out of 6 new replicas have been update
Waiting for deployment "myapp-deployment" rollout to finish: 2 old replicas are pending termination..
Waiting for deployment "myapp-deployment" rollout to finish: 2 old replicas are pending termination..
Waiting for deployment "myapp-deployment" rollout to finish: 2 old replicas are pending termination..
Waiting for deployment "myapp-deployment" rollout to finish: 2 old replicas are pending termination..
Waiting for deployment "myapp-deployment" rollout to finish: 1 old replicas are pending termination..
Waiting for deployment "myapp-deployment" rollout to finish: 1 old replicas are pending termination..
Waiting for deployment "myapp-deployment" rollout to finish: 1 old replicas are pending termination..
Waiting for deployment "myapp-deployment" rollout to finish: 1 old replicas are pending termination..
Waiting for deployment "myapp-deployment" rollout to finish: 5 of 6 updated replicas are available...
Waiting for deployment "myapp-deployment" rollout to finish: 5 of 6 updated replicas are available...
deployment "myapp-deployment" successfully rolled out
```

```
$ kubectl rollout history deployment myapp-deployment
deployment.apps/myapp-deployment
REVISION  CHANGE-CAUSE
1         kubectl.exe create --filename=kube_deployment.yaml --record=true
2         kubectl.exe create --filename=kube_deployment.yaml --record=true
```

- You can see the events of the deployment to exactly debug what had happened during the creation of new revision.
  - **kubectl describe deployment <**deployment name**>**
  - I created a deployment with replicas=3, nginx:1.30

```
$ kubectl rollout status deployment myapp-deployment
deployment "myapp-deployment" successfully rolled out
```

```
$ kubectl rollout history deployment myapp-deployment
deployment.apps/myapp-deployment
REVISION  CHANGE-CAUSE
1         kubectl.exe apply --filename=kube_deployment.yaml --record=true
```

    - One revision is created (**1**).

```
Events:
  Type    Reason             Age   From                   Message
  ----    ------             ----  ----                   -------
  Normal  ScalingReplicaSet  116s  deployment-controller  Scaled up replica set myapp-deployment-748b8f5dcc from 0 to 3
```

    - In the event of deployment object, It scaled up replica from 0 to 3.

```
$ kubectl get rs
NAME                          DESIRED   CURRENT   READY   AGE
myapp-deployment-748b8f5dcc   3         3         3       2m55s
```

    - Currently it created one ReplicaSet.

  - I changed the image: *nginx:1.30-perl*

```
$ kubectl rollout status deployment myapp-deployment
Waiting for deployment "myapp-deployment" rollout to finish: 1 out of 3 new replicas have been updated.
Waiting for deployment "myapp-deployment" rollout to finish: 1 out of 3 new replicas have been updated.
Waiting for deployment "myapp-deployment" rollout to finish: 1 out of 3 new replicas have been updated.
Waiting for deployment "myapp-deployment" rollout to finish: 2 out of 3 new replicas have been updated.
Waiting for deployment "myapp-deployment" rollout to finish: 2 out of 3 new replicas have been updated.
Waiting for deployment "myapp-deployment" rollout to finish: 2 out of 3 new replicas have been updated.
Waiting for deployment "myapp-deployment" rollout to finish: 2 out of 3 new replicas have been updated.
Waiting for deployment "myapp-deployment" rollout to finish: 1 old replicas are pending termination...
Waiting for deployment "myapp-deployment" rollout to finish: 1 old replicas are pending termination...
Waiting for deployment "myapp-deployment" rollout to finish: 1 old replicas are pending termination...
deployment "myapp-deployment" successfully rolled out
```

    - You can see it deleted the previous pods and created new pods.

```
$ kubectl rollout history deployment myapp-deployment
deployment.apps/myapp-deployment
REVISION  CHANGE-CAUSE
1         kubectl.exe apply --filename=kube_deployment.yaml --record=true
2         kubectl.exe apply --filename=kube_deployment.yaml --record=true
```

    - It created one more revision (**2**)

```
Events:
  Type    Reason             Age    From                   Message
  ----    ------             ----   ----                   -------
  Normal  ScalingReplicaSet  7m18s  deployment-controller  Scaled up replica set myapp-deployment-748b8f5dcc from 0 to 3
  Normal  ScalingReplicaSet  2m38s  deployment-controller  Scaled up replica set myapp-deployment-545b99c5d7 from 0 to 1
  Normal  ScalingReplicaSet  2m36s  deployment-controller  Scaled down replica set myapp-deployment-748b8f5dcc from 3 to 2
  Normal  ScalingReplicaSet  2m36s  deployment-controller  Scaled up replica set myapp-deployment-545b99c5d7 from 1 to 2
  Normal  ScalingReplicaSet  2m35s  deployment-controller  Scaled down replica set myapp-deployment-748b8f5dcc from 2 to 1
  Normal  ScalingReplicaSet  2m35s  deployment-controller  Scaled up replica set myapp-deployment-545b99c5d7 from 2 to 3
  Normal  ScalingReplicaSet  2m34s  deployment-controller  Scaled down replica set myapp-deployment-748b8f5dcc from 1 to 0
```

- You can see, its scaling down the pods of previous replicaset (*myapp-deployment-748b8f5dcc*) and scaling up the pods of new replicaset (*myapp-deployment-545b99c5d7*)

```
$ kubectl get rs
NAME                          DESIRED   CURRENT   READY   AGE
myapp-deployment-545b99c5d7   3         3         3       3m38s
myapp-deployment-748b8f5dcc   0         0         0       8m18s
```

- One more ReplicaSet got created.

- I changed the image: *nginx:1,29-perl* using ***set image*** command.

[note, I have written 1,29 not 1.29, so it'll fail]

- **kubectl set image deployment myapp-deployment nginx=nginx:1.30**

  the format is

  kubectl set image deployment <deployment name**> <**container name**>=<**image name**>**

- For me, container name is ***nginx*** because inside ***containers*** (in yaml) I have written nginx

```
spec:
  containers:
    - name: nginx
      image: nginx:1.30
```
(name field)

```
$ kubectl rollout status deployment myapp-deployment
Waiting for deployment "myapp-deployment" rollout to finish: 1 out of 3 new replicas have been updated.
```

- It'll be stucked here because there is no image named *nginx:1,29-perl*

```
$ kubectl rollout history deployment myapp-deployment
deployment.apps/myapp-deployment
REVISION  CHANGE-CAUSE
1         kubectl.exe apply --filename=kube_deployment.yaml --record=true
2         kubectl.exe apply --filename=kube_deployment.yaml --record=true
3         kubectl.exe set image deployment myapp-deployment nginx=nginx:1,29-perl --record=true
```

- It'll create one more revision (**3**).

```
Events:
  Type    Reason             Age    From                   Message
  ----    ------             ----   ----                   -------
  Normal  ScalingReplicaSet  15m    deployment-controller  Scaled up replica set myapp-deployment-748b8f5dcc from 0 to 3
  Normal  ScalingReplicaSet  10m    deployment-controller  Scaled up replica set myapp-deployment-545b99c5d7 from 0 to 1
  Normal  ScalingReplicaSet  10m    deployment-controller  Scaled down replica set myapp-deployment-748b8f5dcc from 3 to 2
  Normal  ScalingReplicaSet  10m    deployment-controller  Scaled up replica set myapp-deployment-545b99c5d7 from 1 to 2
  Normal  ScalingReplicaSet  10m    deployment-controller  Scaled down replica set myapp-deployment-748b8f5dcc from 2 to 1
  Normal  ScalingReplicaSet  10m    deployment-controller  Scaled up replica set myapp-deployment-545b99c5d7 from 2 to 3
  Normal  ScalingReplicaSet  10m    deployment-controller  Scaled down replica set myapp-deployment-748b8f5dcc from 1 to 0
  Normal  ScalingReplicaSet  4m22s  deployment-controller  Scaled up replica set myapp-deployment-8cfcfd55f from 0 to 1
```

- It tried to create new pod but failed as image doesn't exist at all.
- Previous pod was not deleted because already the replica count = 3 and default maxSurge=25% which is 0.75. so deleting one pod will lead to have less number of pod than allowed.

```
$ kubectl get rs
NAME                          DESIRED   CURRENT   READY   AGE
myapp-deployment-545b99c5d7   3         3         3       20m
myapp-deployment-748b8f5dcc   0         0         0       24m
myapp-deployment-8cfcfd55f    1         1         0       13m
```

- Total number of pods = 4 (CURRENT), the new one was not able to be Ready

- Now, we want to rollback to the previous version (**2**) because that was stable.
  - **kubectl rollout undo deployment myapp-deployment**

    format is: **kubectl rollout undo deployment <**deployment name**>**
  - ```
    $ kubectl rollout status deployment myapp-deployment
    deployment "myapp-deployment" successfully rolled out
    ```
  - ```
    $ kubectl rollout history deployment myapp-deployment
    deployment.apps/myapp-deployment
    REVISION  CHANGE-CAUSE
    1         kubectl.exe apply --filename=kube_deployment.yaml --record=true
    3         kubectl.exe set image deployment myapp-deployment nginx=nginx:1,29-perl --record=true
    4         kubectl.exe apply --filename=kube_deployment.yaml --record=true
    ```

    you can see, the 2nd revision is gone, and 4th revision is created which is exact same as the 2nd revision.
  - ```
    Events:
      Type    Reason             Age    From                   Message
      ----    ------             ----   ----                   -------
      Normal  ScalingReplicaSet  31m    deployment-controller  Scaled up replica set myapp-deployment-748b8f5dcc from 0 to 3
      Normal  ScalingReplicaSet  27m    deployment-controller  Scaled up replica set myapp-deployment-545b99c5d7 from 0 to 1
      Normal  ScalingReplicaSet  27m    deployment-controller  Scaled down replica set myapp-deployment-748b8f5dcc from 3 to 2
      Normal  ScalingReplicaSet  27m    deployment-controller  Scaled up replica set myapp-deployment-545b99c5d7 from 1 to 2
      Normal  ScalingReplicaSet  27m    deployment-controller  Scaled down replica set myapp-deployment-748b8f5dcc from 2 to 1
      Normal  ScalingReplicaSet  27m    deployment-controller  Scaled up replica set myapp-deployment-545b99c5d7 from 2 to 3
      Normal  ScalingReplicaSet  27m    deployment-controller  Scaled down replica set myapp-deployment-748b8f5dcc from 1 to 0
      Normal  ScalingReplicaSet  21m    deployment-controller  Scaled up replica set myapp-deployment-8cfcfd55f from 0 to 1
      Normal  ScalingReplicaSet  2m36s  deployment-controller  Scaled down replica set myapp-deployment-8cfcfd55f from 1 to 0
    ```

    You can see, it scaled down the pod of the revision 3's ReplicaSet.
    ```

```
$ kubectl get rs
NAME                          DESIRED   CURRENT   READY   AGE
myapp-deployment-545b99c5d7   3         3         3       28m
myapp-deployment-748b8f5dcc   0         0         0       32m
myapp-deployment-8cfcfd55f    0         0         0       22m
```

- The last ReplicaSet (myapp-deployment-8cfcfd55f) has now 0 Current Pods.
- The existing one was deleted after rolling back.

Command	Description
`kubectl create -f deployment-definition.yml`	Create a new deployment from the YAML definition
`kubectl get deployments`	List all deployments
`kubectl apply -f deployment-definition.yml`	Apply updates to the deployment from the YAML file
`kubectl set image deployment/myapp-deployment nginx-container=nginx:1.9.1`	Update the image for a specific container in the deployment
`kubectl rollout status deployment/myapp-deployment`	Check the status of the rollout
`kubectl rollout undo deployment/myapp-deployment`	Roll back to the previous deployment revision

Kubernetes Networking

- Unlike Docker network where each container is assigned with an IP,
  In Kubernetes each Pod gets an IP, not the container running inside it.
- Kubernetes does not provide a built-in mechanism to resolve IP conflicts in multi-node environments. External networking solutions are required to assign unique IP addresses across all pods.
  - All containers/Pods can communicate to one another without NAT.
  - All nodes can communicate with all containers and vice versa without NAT.
- f

Services

- What is Kubernetes Service?
  - It is a way to expose your Pods (applications) as a stable network endpoint, even though Pods themselves are temporary and can change.
  - In simple term. A permanent entry point that hides the chaos of changing Pods and always routes traffic to the right place.
- Example:
  - Suppose you deployed a Pod running a web application and that pod is present in a node having IP *192.168.1.2*
  - A user is also in the same network having IP *192.168.1.10*
  - The pod IP belongs to the subnet *10.244.0.0/24*

    lets say: *10.244.0.2*
  - User can access the Node (because same network) and Node can access the Pod (because Pod is running within the Node's network).

    But, user can't access the Pod because it is in different network from the User.
  - The user can SSH to the Node and access the Pod. It'll work

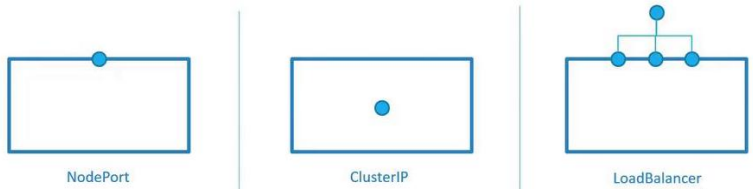
```
curl http://10.244.0.2
Hello World!
```

  - But you don't want this, the user should directly access the application running inside the Pod present inside the Node.
  - This is where Kubernetes Service comes in the picture.

    [Kubernetes service is also an object just like Pod, ReplicaSet, Deployment ..etc.]
  - There is one type of Kubernetes Service which is called ***NodePort Service***, which listens on a specific port of the Node and forwards incoming external traffic to a designated *port* on the *Pod*.
  - Means, it maps a port (eg: 30008) on the Node to the Pod's target port (typically 80 for web servers).
  - For example:

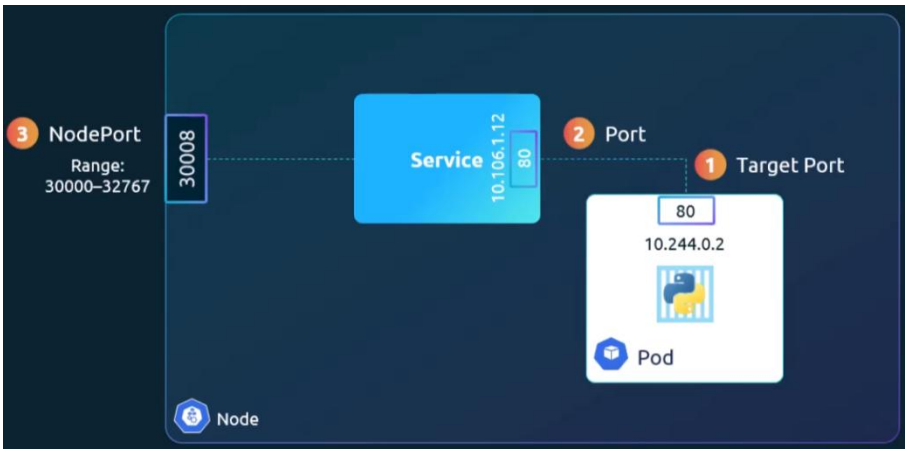
```
curl http://192.168.1.2:30008
Hello World!
```

(it'll work for the user now)

- Types of Services
  - 

  - NodePort Service is the one we discussed previously.

NodePort Service

  - 

  - You can see here, 3 ports are involved.
    - Port of the Pod (80)
    - Port of the Service (80)

      it is **not required** that Pod's port and Service's port will be same.
    - Port of the Node (typically 30000 to 32627)
  - The yaml looks like this

```yaml
apiVersion: v1
kind: Service
metadata:
  name: myapp-service
spec:
  type: NodePort
  ports:
    - targetPort: 80
      port: 80
      nodePort: 30008
```

kind: Service

type: NodePort (type of service)

targetPort (Pod's port)

nodePort (Node's port)

port (Service's port)

- In that key **ports** the only mandatory field is **port** (service's port)

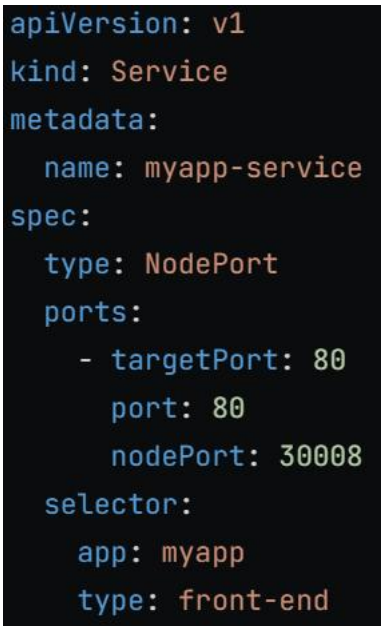
  targetPort and *nodePort* are not mandatory.

  If ***targetPort*** is not provided, it'll be taken same as the ***port***.

  If ***nodePort*** is not provided, it'll take a port in the range *30000 - 32767*

- **NOTE:** **ports** is an array.

➢ But now the question is, how to link the Service to a Pod?

- 

```yaml
apiVersion: v1
kind: Service
metadata:
  name: myapp-service
spec:
  type: NodePort
  ports:
    - targetPort: 80
      port: 80
      nodePort: 30008
  selector:
    app: myapp
    type: front-end
```

```yaml
apiVersion: v1
kind: Pod
metadata:
  name: myapp-pod
  labels:
    app: myapp
    type: front-end
spec:
  containers:
    - name: nginx-container
      image: nginx
```

  - Mark here: in Service yaml, you don't write ***matchLabels*** or ***matchExpressions*** inside ***selector*** unlike *ReplicaSet* or *Deployment*.

- Here, we'll link the pod to Service using ***labels*** and ***selector***

  selector inside Service, and ***labels*** inside Pod.

- If multiple Pods (same type of Pods, same labels ..etc) are present in a Node then NodePort service checks the available routes to which traffic can be forwarded to.

  It chooses one of the pod in random order and route the traffic.

  - If you want one client should always be redirected to a specific Pod (***sticky session***) then sessionAffinity: ClientIP

- In case of Pods across multiple nodes, it opens the same port for each node (eg: 30008) and through which the pods can be accessible.

- You don't need to do anything in case of multiple pods in a single node or multiple pods on different nodes, all will work by itself.

- NOTE:

  - Lets say we have 3 nodes: Node-1, Node-2, Node-3

- The client access to Node-2:30008
- Now, it is not necessary that the request will go to a Pod present inside Node-2 only, it might go to the Pod inside Node-3 as well.
- It is because, ***kube-proxy*** of each node knows about all the Pods inside the cluster (same or other node's pods).

  it can route to anywhere.

- **Service** declares "*traffic to this port should go to these Pods*"; **kube-proxy** programs kernel networking rules *(iptables/IPVS)* so packets are actually redirected to one of those Pods.
  - Service doesn't do routing and all, it just says

    Traffic coming to X should go to one of these Pods on Y port.
  - kube-proxy runs on every node, and watches *services, endpoints (Pods IPs)* and convert that service definition into real packet-handling rules.
  - Example:

```
nodePort: 30008
targetPort: 3000
selector: app=my-app
```

(Service says this)

```
IF packet arrives at :30008
   → choose one Pod IP from:
        10.0.0.2:3000
        10.0.1.3:3000
        10.0.2.4:3000
   → rewrite destination to that Pod IP
   → forward packet
```

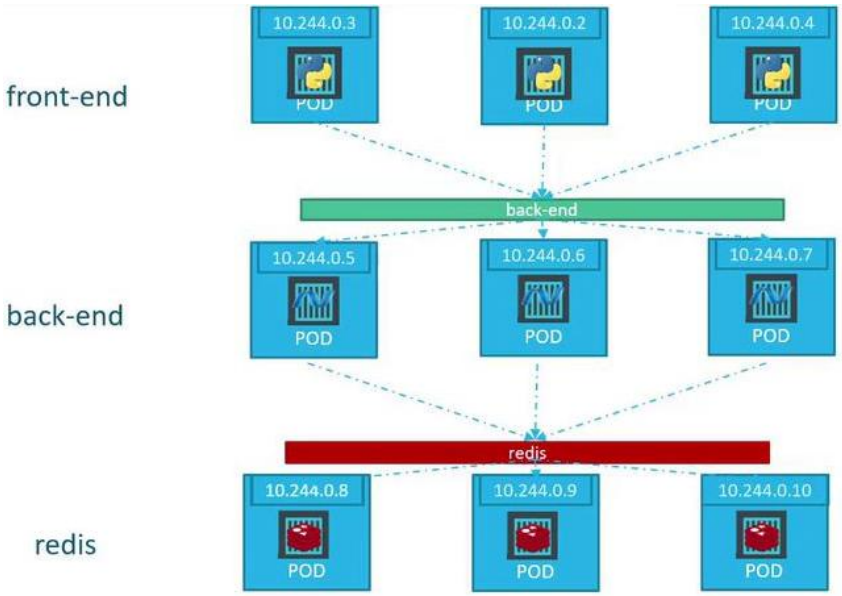
(kube-proxy translates to this)

- Service also has IP

```
$ kubectl describe service myapp-service
Name:                     myapp-service
Namespace:                default
Labels:                   app=myapp-service
Annotations:              <none>
Selector:                 app=myapp
Type:                     NodePort
IP Family Policy:         SingleStack
IP Families:              IPv4
IP:                       10.109.163.119
IPs:                      10.109.163.119
Port:                     <unset>  80/TCP
TargetPort:               80/TCP
NodePort:                 <unset>  30009/TCP
Endpoints:                10.244.0.120:80,10.244.0.118:80,10.244.0.119:80
Session Affinity:         None
External Traffic Policy:  Cluster
Internal Traffic Policy:  Cluster
Events:                   <none>
```

➢ ClusterIP Service

- Inside your Kubernetes cluster, there can be *n* types of Pods running (n types, not n numbers)
  - Like, frontend pods, backend pods, redis pods, DB pods ..etc.
- There is need of one type of pods communicating with another type of pods.
  Eg: frontend pods need to communicate with backend pods, similarly backend pods needs to communicate with redis pods or DB pods …etc.



- **ClusterIP** is the default service, i.e. if you omit the *type* field inside the yaml, then it'll take it as ClusterIP by default.

```yaml
apiVersion: v1
kind: Service
metadata:
  name: back-end
spec:
  type: ClusterIP
  ports:
    - port: 80
      targetPort: 80
  selector:
    app: myapp
    type: back-end
```

```yaml
apiVersion: v1
kind: Pod
metadata:
  name: myapp-pod
  labels:
    app: myapp
    type: back-end
spec:
  containers:
    - name: nginx-container
      image: nginx
```

- Here *nodePort* is not there unlike NodePort Service.

- Everything looks same, means the selector is also only one, not more.
  So it is also applied to same type of Pods. What is the difference between ClusterIP service and NodePort service then?
  - NodePort service opens a port on each Node and select one type of pod to which external traffic is forwarded to the Pod.
  - Whereas *ClusterIP* service is used for internal communication between pods.
    It doesn't open any port in the Nodes for external communication.
  - The flow in *ClusterIP* is like

```
Pod → ServiceIP:80 → Pod
```

```
Pod (client)
   ↓
ServiceIP:80   ← (this is "port")
   ↓
kube-proxy
   ↓
PodIP:80       ← (this is "targetPort")
```

- If *port* and *targetPort* are different

```
ports:
  - port: 80
    targetPort: 3000
```

```
Pod → ServiceIP:80 → PodIP:3000
```

- When you open the details of a service (**kubectl describe service <**service name**>**), you'll see one thing: **Endpoints**

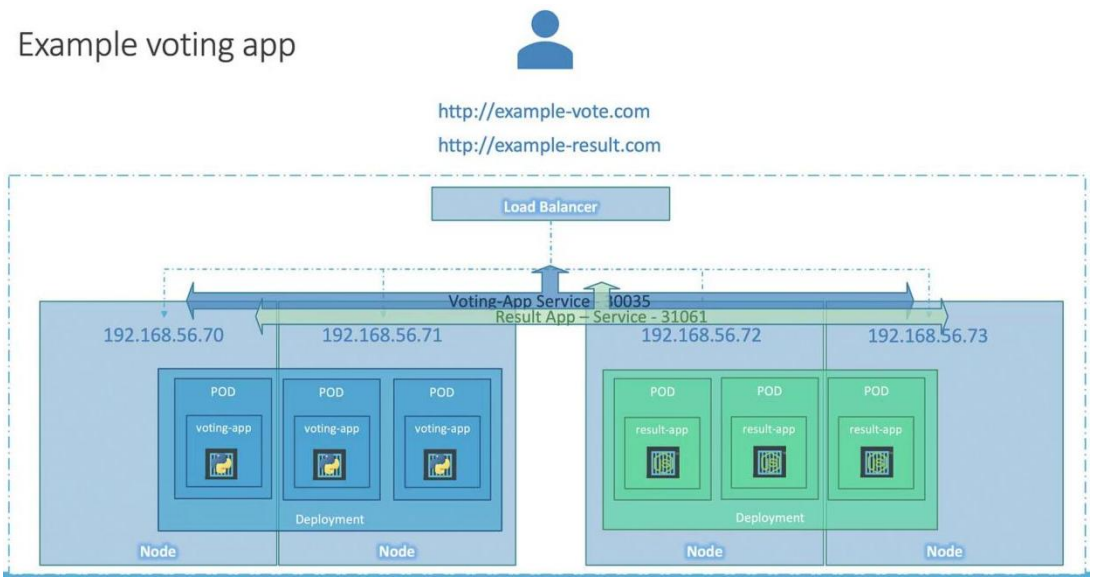
```
$ kubectl describe svc myapp-service
Name:                     myapp-service
Namespace:                default
Labels:                   app=myapp-service
Annotations:              <none>
Selector:                 app=myapp
Type:                     NodePort
IP Family Policy:         SingleStack
IP Families:              IPv4
IP:                       10.109.163.119
IPs:                      10.109.163.119
Port:                     <unset>  80/TCP
TargetPort:               80/TCP
NodePort:                 <unset>  30009/TCP
Endpoints:                10.244.0.120:80,10.244.0.118:80,10.244.0.119:80
Session Affinity:         None
External Traffic Policy:  Cluster
Internal Traffic Policy:  Cluster
Events:                   <none>
```

it shows all the pods IPs with the targetPort which are there in the cluster to which the client's traffic can be forwarded to. (3 pods are available so 3 entries)

```
$ kubectl get pod -o wide
NAME                                READY   STATUS    RESTARTS   AGE   IP             NODE       NOMINATED NODE   READINESS GATES
myapp-deployment-545b99c5d7-cvmhs   1/1     Running   0          23h   10.244.0.118   minikube   <none>           <none>
myapp-deployment-545b99c5d7-jkfgc   1/1     Running   0          23h   10.244.0.120   minikube   <none>           <none>
myapp-deployment-545b99c5d7-qbpbn   1/1     Running   0          23h   10.244.0.119   minikube   <none>           <none>
```

(you can check the IPs of the Pods, those are present in the *Endpoints* field.)

➢ LoadBalancer Service



- You can see in this image, we have 2 types of pods.

  Voting-app and *result-app*.
- So, 2 Deployments are there, and 2 types of NodePort service.
- 4 nodes are there and 6 pods are present across the nodes.
- Node's IPs are: *192.168.56.70, 192.168.56.71, 192.168.56.72, 192.168.56.73*
- We have 2 NodePort services, so 2 ports which are *30035* and *31061*.
- So, we have now 8 combinations. Each node's IPs with 2 ports.
- Now you can say, *voting-app* is there on node-1 and node-2, *result-app* is there on node-3 and node-4. then why do we need to bind the node-1, node-2 with the port 31061 which is of result-app and same for node-3 and node-4 for port 30035.
  - Now, when you create a NodePort service, it applies on all the nodes of the cluster.
  - Now the *voting-app* pods are present in node-1 and node-2, but these are temporary and can be present on node-3 or node-4 in future.
  - So, from any node you can access any Pod in the cluster. That pod might be present in the same node or other.

    Inside the cluster, all pods are present in the same network, that doesn't depend on the node.

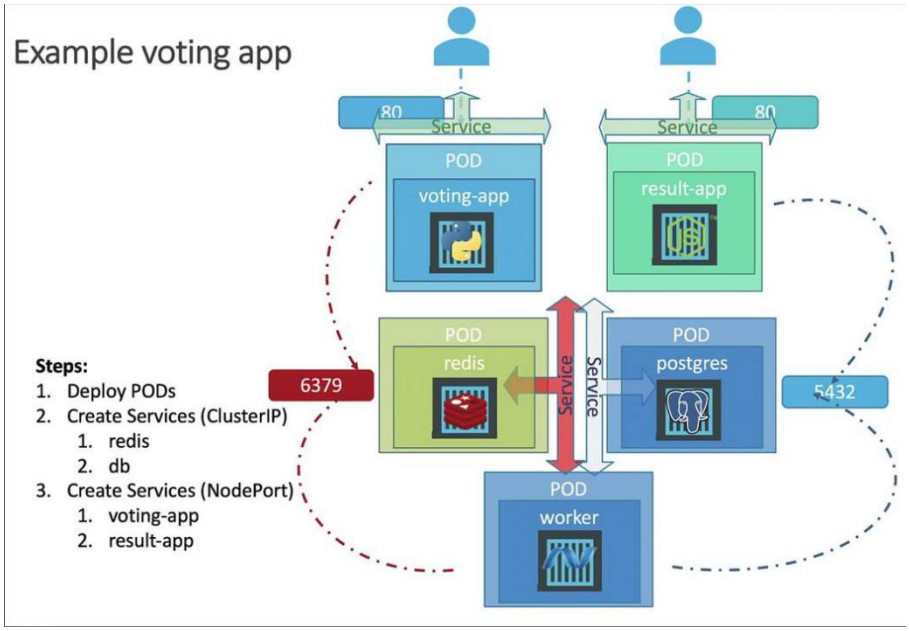
    [just node gets subnet from which pods get their IPs, but after all, all the pods fall under the same network]
- Now, you need a load balancer that will route the traffic to a specific *node:port* combination.

```yaml
apiVersion: v1
kind: Service
metadata:
  name: myapp-service
spec:
  type: LoadBalancer
  ports:
    - targetPort: 80
      port: 80
      nodePort: 30008
```

- In cloud services like AWS, Azure, GCP, they are having the load balancer. So, in that case kubernetes use that load balancer only.
- In VM, you need to use nginx or NodePort service only.

➢

Deploying a Microservice Architecture in Kubernetes

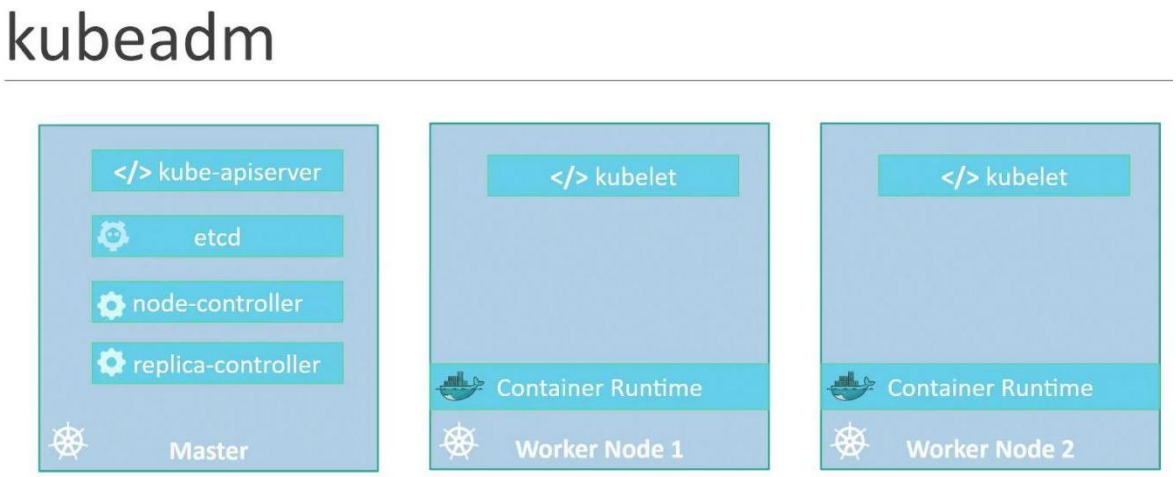
- 

  - This is our application's format.
  - 2 apps are there voting-app and result-app that user can access.
  - After the user votes in *voting-app* it updates the *redis* cache.
  - Worker takes the data from *redis* and store in the persistent data storage which is *postgres* in this case.
  - *result-app* access the db and show the result to the users.
- First we need to map which pods are being accessed by others.
  - Worker is not accessed by anyone, so it doesn't depend on any other.
  - *redis* is accessed by both *worker* and *voting-app*.
  - *postgres* is accessed by both *worker* and *result-app*.
- Hostname mappings
  - The postgres db is accessed with hostname **db** inside the application, so the postgres DB's ClusterIP service name must be ***db***.
  - Similarly for ***redis*** Service also, the service name must be ***redis***.
-

kubeadm tool

- What is **kubeadm**?
  - If you want to run Kubernetes then you need to setup all the things manually like generating certificates, setup control plane services (kube-controller-manager, etcd, kube-api-server, kube-scheduler) and setup kubelet, k-proxy ..etc in worker nodes.
  - ***Kubeadm*** automates essential tasks such as *component installation, configuration file setup, and secure certificate generation,* ensuring your *multi-node* cluster adheres to Kubernetes best practices.

    [etcd is tightly coupled with control plane so it is often considered it as a control plane component. It is the *distributed* key-value pair storage or you can say memory which stores every details about the cluster. If it goes down, cluster will be down. *etcd* is distributed among all the *master* nodes which work in sync to make *etcd* a single entity]



- Requirements:
  - First you need to have multiple systems or VMs to configure the cluster.
  - Then designate one node as master and other 2 as worker nodes.
  - Install a container runtime on the hosts (VMs).

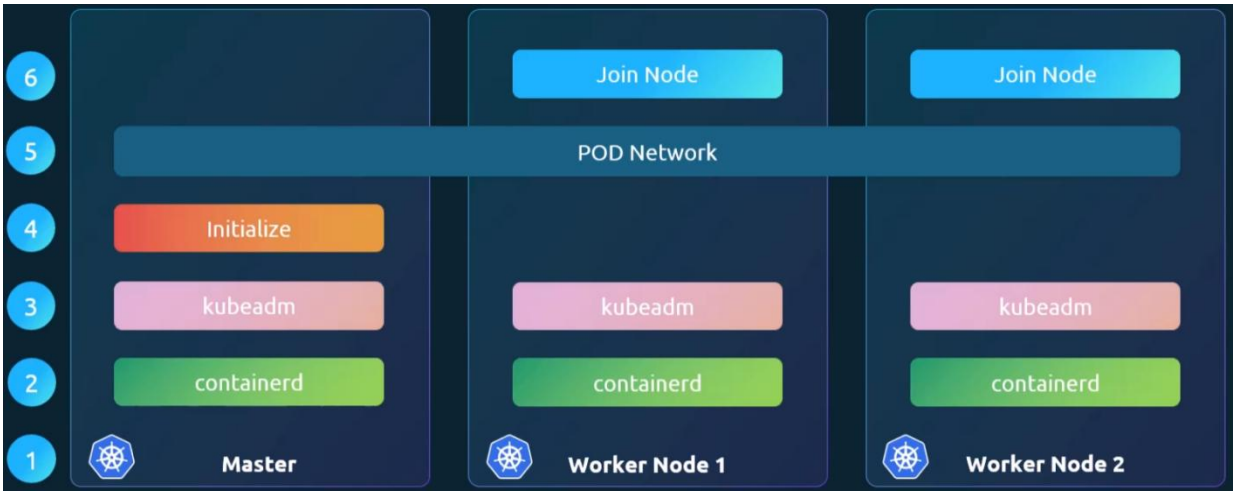
    we'll be using Docker so need to install Docker in all the nodes.
  - Install **kubeadm** tool on all the nodes.
  - Initialize the master server.

    During this process all the required configurations will be installed and configured on the master server.
  - Once master is initialized and before joining the worker nodes to the cluster, you must ensure the network prerequisites are met.

Kubernetes requires a special networking solution between the master and worker nodes which is called as POD Network.

- Then join the *worker nodes* with the *master node*.

  Now ALL SET.



- To setup the things inside the VMs, you can go through this https://github.com/kodekloudhub/certified-kubernetes-administrator-course/tree/master/kubeadm-clusters/generic

CGROUP

- What is **cgroup**?
  - **cgroup** is Control Group.
  - It is a linux kernel feature to control and limit the resources used by processes.
    It let you limit CPU, memory, Disk I/O ..etc.
  - **cgroupfs** is nothing but abbreviation of *Control Group File System* which is mounted to the path ***/sys/fs/cgroup*** .
    - You can't see the file system mounted to /sys/fs/cgroup using *df -h* command because it is not a normal disk filesystem.
      df -h typically display disk-based file systems.
    - **cgroupfs** is a ***virtual kernel based*** filesystem.
    - You can see the mount using the command ***mount | grep cgroup*** (mount command display all types of mounts)
      you'll see something like this

      ```
      cgroup2 on /sys/fs/cgroup type cgroup2 (rw,nosuid,nodev,noexec,relatime,nsdelegate,memory_recursiveprot)
      ```

      in older systems, you'll see ***cgroup*** instead of ***cgroup2***.
    - Here **cgroup2** is nothing but a type of file system just like */dev/sda1* or *tempfs* ..etc that you get when you execute ***df -h***

      ```
      vagrant@controlplane:/sys/fs/cgroup$ sudo df -h
      Filesystem      Size  Used Avail Use% Mounted on
      tmpfs           197M  1.1M  196M   1% /run
      /dev/sda1        39G  2.0G   37G   6% /
      tmpfs           982M     0  982M   0% /dev/shm
      tmpfs           5.0M     0  5.0M   0% /run/lock
      vagrant         953G  438G  515G  46% /vagrant
      tmpfs           197M  4.0K  197M   1% /run/user/1000
      ```

- Directories vs Files inside ***/sys/fs/cgroup***

  ```
  vagrant@controlplane:/sys/fs/cgroup$ ls
  cgroup.controllers      cpuset.mems.effective  memory.pressure
  cgroup.max.depth        dev-hugepages.mount    memory.stat
  cgroup.max.descendants  dev-mqueue.mount       misc.capacity
  cgroup.procs            init.scope             proc-sys-fs-binfmt_misc.mount
  cgroup.stat             io.cost.model          sys-fs-fuse-connections.mount
  cgroup.subtree_control  io.cost.qos            sys-kernel-config.mount
  cgroup.threads          io.pressure            sys-kernel-debug.mount
  cpu.pressure            io.prio.class          sys-kernel-tracing.mount
  cpu.stat                io.stat                system.slice
  cpuset.cpus.effective   memory.numa_stat       user.slice
  ```

  - All the files (not directories) present inside this directory are *created and handled* by Linux Kernel.

- All cgroup.* , cpu.* , memory.* , io.* are created and handled by Kernel itself.

- *Directories* with suffix **.slice** **.scope** **.service** **.mount** are managed by ***systemd***.
- *Files* like **cpu.*** **memory.*** **cgroup.*** **io.*** are managed by Kernel.
- Other directories like **docker/** **kubepods/** (if present) are managed by their respective owner.
- Inside that ***system.slice*** all the system services (background daemons) like *ssh.service, docker.service, cron.service* are there.

```
vagrant@controlplane:/sys/fs/cgroup/system.slice$ ls
 cgroup.controllers       cpuset.mems             multipathd.service
 cgroup.events            cpuset.mems.effective   networkd-dispatcher.service
 cgroup.freeze            cron.service            packagekit.service
 cgroup.kill              dbus.service            pids.current
 cgroup.max.depth         io.max                  pids.events
 cgroup.max.descendants   io.pressure             pids.max
 cgroup.procs             io.prio.class           polkit.service
 cgroup.stat              io.stat                 rsyslog.service
 cgroup.subtree_control   io.weight               snap-core20-2717.mount
 cgroup.threads           irqbalance.service      snap-lxd-38469.mount
 cgroup.type              memory.current          snap-snapd-26382.mount
 cloud-final.service      memory.events           snapd.service
 containerd.service       memory.events.local     snapd.socket
 cpu.idle                 memory.high             ssh.service
 cpu.max                  memory.low              system-getty.slice
```

  - These are started by system (boot or service manager)
  - Not tied to any logged-in user.
  - You can see *containerd.service* which is for containerd (container runtime).
- Whereas ***user.slice*** contains the details about the user like user sessions and processes.

➢ How the things are handled by Kernels?

- The managers (eg: systemd, docker(if present) ..etc) write into their respective directories.
- Kernel enforces the limits.
- `Docker → writes to /sys/fs/cgroup/docker/... → kernel enforces limits`

  (in case of Docker's own cgroup, it'll be maintained by Docker)
- In real, docker does the below.

  it write limits inside files like *memory.max*, *cpu.max* and track processes *cgroup.procs* present inside the below directory

  `/sys/fs/cgroup/docker/<container-id>/`

  kernel applies memory limit to that container.

- But in this case, 2 managers are there (Docker, systemd)
  both are independently managing their resources, no co-ordination.
  - Lets say Docker containers are using 6GB of memory.
  - Systemd sees, its processes are only using 2GB of memory.
- Now, Kernel triggers OOM (Out Of Memory) and it must *kill* something.
  - It may kill some critical resources.

➢ To configure ***containerd*** to use ***systemd*** as the *cgroup driver* then below is the command

- ```
  sudo mkdir -p /etc/containerd

  containerd config default | \
  sed 's/SystemdCgroup = false/SystemdCgroup = true/' | \
  sudo tee /etc/containerd/config.toml
  ```
- ***tee*** is just like *output redirection*, but *tee* also print the output in the terminal along with sending that to the file.
- The command is just updating */etc/containerd/config.toml* to use SystemdCgroup as its cgroup driver.
  ```

Some Important Points

- If you are having multiple types of services to deploy like frontend, backend …etc, you need to create multiple Pod or ReplicaSet or Deployment yamls for each type of service.
- **ReplicationController** and **ReplicaSet** has one difference that is ***selector*** field.
  ReplicationController is old, ReplicaSet is modern.
- **deployment.apps/myapp-deployment** is same as **deployment myapp-deployment**
  [deployment object's name is 'myapp-deployment' in this case]
- NodePort Service is used for external traffic to pod forwarding.
  ClusterIP Service is used for pod to pod communication. [i.e. frontend pod to backend pod .. etc]
- In case of NodePort service, if multiple pods are present in multiple nodes, and the client tries to access the port with one of those node, then it is not necessary that the pod to which client's traffic will be forwarded to would be of same node to which the client sent the request.
  - NodePort is not specific to a node, its through out the cluster for one type of Pod [like frontend pods, backend pods ..etc].
- If you have 2 types of pods across 4 nodes, then you need to create 2 Deployments (for each type of pod) and 2 NodePort service needs to be created.
  So, you'll have combination of 8 now
  4 nodes with one port (1st type of pod)
  4 nodes with another port (2nd type of pod)
- In case of ReplicaSet or Deployment, inside selector you write a key like ***matchLabels*** or ***matchExpressions***,
  but inside service, you write the label's keys and values directly inside ***selector*** field.
- ***Deployment = ReplicaSet +*** [rollout, rollback, versioning(revisions), rollout strategy (Recreate, RollingUpdate), Pause, Resume]
  each triggers (image change, env change, ..etc) create a new revision.
  each revisions create one new replicaset so new pods as well.
- Lets say you are deploying a microservice architecture, and suppose 2 apps are there, one is backend and another is postgres database.
  Inside your application, you have written the postgres db url like **db://…**
  now, when you'll deploy your backend application and postgres db in kubernetes, the backend application will continue to use **db** as the hostname of the postgres

database.

And, in kubernetes, you need to have ClusterIP service so that backend application can talk to DB pod.

Now, that ClusterIP Service's name must be **db** (metadata.name), if you give something else then your application will break.

The hostname used in your application must match the Service name.

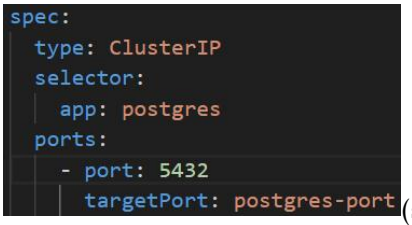
- You can specify a ***name*** of the ***containerPort*** inside Pod which can be used as an alias for that specific port.

```
containers:
  - name: app
    image: my-app
    ports:
      - name: http
        containerPort: 3000
```
(Pod or Deployment)

```
spec:
  selector:
    app: my-app
  ports:
    - port: 80
      targetPort: http
```
(here ***http*** means ***3000*** as it is mentioned in that Pod/Deployment)

  - It means service's 80 port will be connected to pod's 3000 port.

- ***ports*** field inside Pod doesn't expose those port. **IMOPRTANT**
  - Even without giving those port the app will work properly because when you create Service you define the ***targetPort*** .
    through this it calls the Pod with that *targetPort*.
  - ***ports*** is given for user's documentation,
    or, named *containerPort* to be used.
    Or, help tools like kubectl, dashboards.
    - When you call kubectl describe pod <pod-name>, it displays the info that the Pod is using some xyz port.
  - Named port means like this
    - ```
      ports:
        - containerPort: 5432
          name: postgres-port
      ```
      (pod)
      ```

- 

(service, postgres-port will be same as 5432 as mentioned in the *containerPort* of Pod)
- Inside ***containers*** (inside Pod's *spec*), for each container ***name*** field is required which is used to uniquely identify the container inside the Pod.
  (one Pod might have multiple containers running)

- **NodePort** type of Service can also be used for internal (Pod to Pod) communication but not preferred (because it contains port and targetPort; for external traffic *nodePort* is there).
  For internal communication, **ClusterIP** service is preferred.
- In service you can see **ports** is a list (not a single port) where you can give *n* numbers of *port*, *targetPort*, (and in case of NodePort service, *nodePort*).
  - If there are multiple containers running inside same Pod, then all the containers will be having ***same IP, namespace, cgroup***.
  - The only difference will be port.
    Think about your laptop, if you host multiple websites, then you need to give different port (just assume Pod as your local system).
  - Inside the Pod, containers can talk to each other with *localhost* and respective *ports*.
  - So, in the *service yaml*, you need to mention all the target ports of the target Pod (all the ports of the containers running inside the target Pod).

```
ports:
  - name: redis-port
    port: 6379
    targetPort: 6379
  - name: sidecar-port
    port: 9000
    targetPort: 9000
```

  - Now,
    <service name>:6379 => it'll go to the redis container in that Pod.
    <service name>:9000 => it'll go to the sidecar container in that same Pod.
-

Networking

- How a Pod gets IP?
  - First, cluster is created with CIDR (e.g. 10.244.0.0/16)
  - When a node joins the cluster, it gets a subnet (PodCIDR)

    Example: Node A → 10.244.1.0/24

    [assigned by **kube-controller-manager** OR by **CNI** like Calico]
  - When a Pod object is created, scheduler assigns a node.
  - ***kubelet*** on that node detects the Pod.

    It tells container runtime (e.g. containerd) to: "Create ***PodSandbox***"
  - container runtime:
    - creates ***PodSandbox*** (network namespace created)
    - calls CNI plugin (CNI ADD)
  - Now, CNI plugin:
    - Assigns IP (via IPAM)
    - Creates veth pair
    - Attaches to ***bridge*** OR ***sets up routing***
  - CNI returns the IP to container runtime.
  - container runtime:
    - configures PodSandbox with that network
    - starts containers inside the Pod.
  - Kubelet updates the Pod status (running).
- Cluster Created with CIDR, what does that mean?
  - It means you define a pool of IPs for all Pods in the cluster.
  - Ex: 10.244.0.0/16
  - This is configured at cluster setup time (not dynamically per pod)
  - Set via control plane.
  - Who assigns it?

    Its YOU (or) Cluster setup tool (kubeadm, etc.)
- Bridged
  - A bridge is a non-intelligent device which has some ports to which multiple systems can connect to.

    And it broadcast the traffic to all the ports.

    It is the OLD bridge, modern bridge can learn the MAC and maintains a MAC table.

- The Node (where *kubectl*, *containerd* are running).

  like AWS EC2, or any VM ..etc
- CNI Plugin creates a virtual Bridge inside the Host (linux server or node)

  this is called ***cni0*** .

  You can see it using the command **ip link**

  ```
  cni0: <BROADCAST,MULTICAST,UP>
  ```
- Linux bridge is Layer-2 switch, which learns the MAC address and send the traffic to the specific port instead of broadcasting that.
- Now in the bridge (**cni0**) there are multiple veth (virtual eth) ports like.

  ```
  cni0 (bridge)
    ├── veth-host-A
    ├── veth-host-B
    ├── veth-host-C
  ```
- Now, the veth link will be created betwee Pod and cni0

  Pod eth0 --- veth-pod --- veth-host-A --- cni0
  - **Pod-eth0**, and **cni0** can be directly connected. But its ***not possible*** because both are in different namespace. Have to see why and how

➢ How Pods communicate with each other?

- Pods inside same node
  - Pods within the same node doesn't go through Routing (Layer 3)

    they communicate via ***cni0*** bridge (Layer 2 switching)

    ```
    Pod A
      ↓
    eth0   (this is the veth endpoint inside pod)
      ↓
    veth-host (Node)
      ↓
    cni0 (bridge)
      ↓
    veth-host (Pod B)
      ↓
    eth0   (inside Pod B)
      ↓
    Pod B
    ```
  - veth-pod and eth0 are same interface only
- Pods inside different nodes

- ```
  Node A:
    Pod A → 10.244.1.5

  Node B:
    Pod B → 10.244.2.6
  ```
  (Pod's IPs)
- ```
  Node A → 192.168.1.10
  Node B → 192.168.1.20
  ```
  (Node's IPs)
- ```
  Pod A
    ↓
  eth0 (10.244.1.5)
    ↓
  veth-host (Node A)
    ↓
  cni0 (bridge, Node A)
    ↓
  Node A routing table  ← (decides next hop)
    ↓
  eth0 (Node A → 192.168.1.10)
    ↓
  Network
    ↓
  eth0 (Node B → 192.168.1.20)
    ↓
  Node B routing
    ↓
  cni0 (Node B)
    ↓
  veth-host
    ↓
  eth0 (10.244.2.6)
    ↓
  Pod B
  ```
- Here the Layer 3 (Node's Kernel) has to be introduced which maintains the IP table.
- In case of pods inside the same node, ***cni0*** bridge was enough to forward the traffic, but here it is in different subnet (because different node)
- Inside the Node, ***cni0*** (bridge by CNI Plugin) and IP table (that linux kernel maintains) are there.
  ```

- Routing entry of Node A

  `10.244.2.0/24 → via 192.168.1.20` (10.244.2.0/24 subnet is for Node-B, whose IP is 192.168.1.20)

  [**NOTE**: Node's IP and Its subnet is not same. Subnet's IPs are only for Pods, not for the Node itself]

- Routing entry of Node B

  `10.244.1.0/24 → via 192.168.1.10`

- This routing entries are set by CNI Plugins.

➢

➢ F

Basic Networking

- Your laptop can have 2 IPs at once.
  -
    ```
    Interface: WiFi
      IP: 192.168.1.10
      MAC: AA:AA:AA

    Interface: Ethernet
      IP: 10.0.0.5
      MAC: BB:BB:BB
    ```
    - one is wifi and another s ethernet.
  - Now, you'll have 2 IPs.
    ```
    WiFi → 192.168.1.10 (via Router A)
    Ethernet → 10.0.0.5   (via Router B)
    ```
  - You now exists in 2 different networks at the same time.
  - Now if you send a request, it doesn't go through both the interface.

    System chooses one interface, and from that interface the traffic will go.
  - Even though you have 2 IPs, a single request uses only ONE IP/interface
  - Every network interface has a MAC, and an IP is assigned to that interface.
  - If you connects to 2 broadbands with your 2 ethernet interfaces (or with USB-to-Ethernet adaptor), you are connected to 2 routers.

    Now you have 2 active interfaces. (eth-0, eth-1)
- How traffic is sent through IP and MAC in real world?
  - Lets say you opened a browser and typed 'google.com'.

    google server IP: 140.250.x.x (let)
  - Your setup:
    ```
    Your Laptop
    IP: 192.168.1.10
    MAC: AA:AA:AA

    WiFi Router
    IP: 192.168.1.1
    MAC: RR:RR:RR
    ```
  - Now your laptop compares both IPs and decides if they are in same network or not.

    Source IP: *192.168.1.x*

Dest IP: *140.255.x.x*

Noy Same, It needs a Router.

- Now Laptop will needs the Router's MAC to which it is connected to. (it'll use the interface that is being chosen if it is connected to multiple network, here assuming the laptop is connected to a single network only)

  Now, Laptop will shout, who has the IP '*192.168.1.1*' (ARP Request)

  Router replies: Meeeeeeeee.., my MAC is: *RR:RR:RR*

- Now, laptop will send the first packet:

```
Frame (Layer 2):
  SRC MAC: AA:AA:AA
  DST MAC: RR:RR:RR   ← router

Packet inside (Layer 3):
  SRC IP: 192.168.1.10
  DST IP: 142.250.x.x ← Google
```

  NOTE:

  MAC: router (next hop)

  IP: final destination (Google)

- Now, router receive the packet, and see the MAC matched.

  It says: Yayyyy, this packet is for meeeee.

  Now, the router needs to forward the packet to internet.

- Router creates a new frame.

```
SRC MAC: Router's ISP interface MAC
DST MAC: ISP router MAC

SRC IP: 192.168.1.10
DST IP: 142.250.x.x   ← SAME
```

  NOTE:

  only the MACs are changing hop to hop.

  But, IPs are fixed.

- This repeats.

  Every router does the below

  - Removed Old MAC
  - Check IP
  - Add New MAC

- Now, at the last Router.

  It says, who has '*142.250.x.x*'

Google replies: Meeeeeeeeee, and ***replies with its MAC***.

```
SRC MAC: Last router
DST MAC: Google server

SRC IP: 192.168.1.10
DST IP: 142.250.x.x
```

- Now, Google server receives the request.

➢ As we discussed about the packet flow from source to destination, in between Router to Router packet transfer also happened. How does that happen?

- Router to Router also same process.

  Router-A and Router-B (lets say)
- Router-A sends ARP request to Router-B.

  Router-B replies with its MAC.

  [then the MAC (Source and Destination) will be updated, but IPs will be fixed as we saw previously]
- But, question is, how Router-A knows that it needs to forwards the traffic to Router-B? it might have been connected to so many other routers.
  - Routers doesn't guess the next hop.

    It already knows from its *Routing Table*.
  - In every router, the router table will be like this

Destination Network	Next Hop	Interface
192.168.1.0/24	DIRECT	eth0
10.0.0.0/24	DIRECT	eth1
0.0.0.0/0	10.0.0.2	eth1

    that 0.0.0.0/0 is the default GW, it nothing is found then send the traffic to that one.
  - Now, it matches the IP (142.250.x.x).

    nothing matches, so it falls to

    0.0.0.0/0 10.0.0.2
  - Now, Router-B (10.0.0.2) replies with its MAC.

    Router-A will update the packet's MAC addresses, and then this process goes on.

- What is Gateway and Default Gateway?

Destination Network	Next Hop	Interface
192.168.1.0/24	DIRECT	eth0
10.0.0.0/24	DIRECT	eth1
0.0.0.0/0	10.0.0.2	eth1

  - You can see here, the IPs that are present already like

    192.168.1.0/24

    10.0.0.0/24

    these are the Gateways. If any request comes to this router that contains the destination IP among these 2, then it'll forward the traffic to the router with that IP .
  - When it doesn't find the destination IP in its routing table, it falls under default gateway which is ***0.0.0.0/0***

    it sends the traffic to this IP (here it is 10.0.0.2)

Debugging Tips

- If the container is not running inside the pods, then see the events at the end after executing the command

  kubectl describe pod <pod name**>**

  - Inspect the pod's events to get to know about the container if it is not running. The events of replicaset or deployment will not help in this case.

- F

Commands

- **kubectl get nodes -o wide**

```
controlplane ~ ➜ kubectl get nodes -o wide
NAME           STATUS   ROLES           AGE   VERSION        INTERNAL-IP      EXTERNAL-IP   OS-IMAGE             KERNEL-VERSION   CONTAINER-RUNTIME
controlplane   Ready    control-plane   30m   v1.35.0+k3s3   192.168.18.136   <none>        Alpine Linux v3.16   6.8.0-1047-gcp   containerd://1.6.8

controlplane ~ ➜ kubectl get nodes
NAME           STATUS   ROLES           AGE   VERSION
controlplane   Ready    control-plane   30m   v1.35.0+k3s3
```

it gives extra details about the nodes.

- You can see the OS-IMAGE using

  cat /etc/os-release

  but, for this you need to have access to that node or you need to do ssh to that node and do this.

- On the other hand, **kubectl** command has all the nodes registered and without ssh or something else it can provide you the details.

-